Handling Missing Data

Python Data Science Handbook — Chapter 3, Section 3.4

None vs. NaN, dtype upcasting, and the `isnull`/`dropna`/`fillna` toolkit.

1 Why "missing" is a first-class problem

Real data has holes. A sensor drops a reading, a survey respondent skips a question, two tables get joined and one side has no matching row. Before we can compute averages or fit models, we need a principled way to say "there is no value here" — and to make every downstream operation behave sensibly when it meets one.

Background & Context

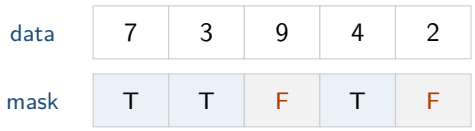
There are two broad strategies for marking a missing entry, and essentially every data system picks one (or pays to support both).

- **Mask approach.** Keep the data array exactly as it is, and store a *separate* boolean array of the same shape: `True` where a value is present, `False` where it is missing. The data array's values at the missing positions are simply ignored.
- **Sentinel approach.** Reserve one specific value inside the data itself to mean "missing." For floating-point data the natural choice is the IEEE special pattern `NaN`; for other types one might steal a value like `-9999` or a bit pattern, and agree by convention that it denotes a hole.

Each has a real cost. The **mask** costs extra memory (a whole parallel boolean array) and extra bookkeeping (every operation must consult the mask). The **sentinel** costs you a value from the type's range — whatever bit pattern you reserve can no longer be a legitimate datum — and it is *dtype-specific*: a float can spare a `NaN`, but a plain machine integer has no spare bit pattern to give up.

Pandas was originally built on top of NumPy, and NumPy arrays have no native mask. So historically Pandas leaned on the **sentinel** strategy, reusing whatever sentinels the underlying types already offered: Python's `None` object and the floating-point `NaN`. The rest of this section is really the story of those two sentinels and the machinery Pandas wraps around them.

Mask approach



data untouched; a parallel boolean array says which cells count. Costs an extra array.

Sentinel approach

one reserved value lives *inside* the array and means "missing." No extra memory, but you give up a usable value and it is type-specific.

2 Sentinel #1: Python's `None`

Python already ships a universal "nothing here" object: the singleton `None`. It is the obvious first candidate for a sentinel, and Pandas does accept it. But putting `None` into a NumPy array has an important consequence.

```python
import numpy as np
import pandas as pd

vals = np.array([1, None, 3, 4])
print(vals)
print(vals.dtype)
```

```
[1 None 3 4]
object
```

Key Idea

NumPy needs every element of an array to share one dtype. A machine integer and the Python object `None` have no common *numeric* type, so NumPy falls back to the only type that can hold anything: `dtype=object`. An object array is really an array of pointers to full Python objects.

That fallback is expensive in two distinct ways.

Speed. An `object` array cannot use NumPy's fast, vectorized, C-level loops. Every operation has to dispatch back into the Python interpreter, element by element — the same overhead you were trying to escape by using NumPy in the first place. A sum over a million-element object array can be an order of magnitude or more slower than over a native `int64` array.

Broken aggregations. Worse, common reductions simply fail:

```python
np.array([1, None, 3]).sum()
```

```
TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

Common Pitfall

`None` is not a number, and Python has no definition for `int + None`. So a `sum()` over an object array containing `None` raises `TypeError` rather than skipping the hole. `None` is a fine sentinel for "a Python object is absent," but a poor one for "a numeric value is absent," precisely because arithmetic on it is undefined.

3 Sentinel #2: the floating-point `NaN`

The numeric world has its own purpose-built sentinel: `NaN`, "Not a Number," a special bit pattern defined by the IEEE-754 floating-point standard. Crucially it *is* a float, so it lives happily inside a native float array with no dtype downgrade.

```python
vals = np.array([1, np.nan, 3, 4])
print(vals.dtype)   # → float64  (a fast, native array)
```

```
float64
```

Because the array stays `float64`, all the fast vectorized machinery still works. The catch is a different one: `NaN` is *contagious*.

```
print(1 + np.nan)           # → nan
print(0 * np.nan)           # → nan
print(np.array([1, np.nan, 3]).sum())    # → nan
```

```
nan
nan
nan
```

Intuition

Think of `NaN` as a stain that spreads. By design, *any* arithmetic that touches a `NaN` produces a `NaN` — $1 + \text{NaN} = \text{NaN}$, $0 \times \text{NaN} = \text{NaN}$, and so a plain `sum` of a column with one hole is itself `NaN`. This is mathematically honest (the answer truly is undefined if one input is unknown), but it means a single missing cell can silently nullify an entire aggregate.

The fix is to use *NaN-aware* reductions, which skip the holes instead of propagating them:

```
x = np.array([1, np.nan, 3, 4])
print(np.nansum(x))     # → 8.0
print(np.nanmean(x))    # → 2.6666666666666665
print(np.nanmax(x))     # → 4.0
```

```
8.0
2.6666666666666665
4.0
```

Python Refresher

`NaN` also breaks one rule you may rely on: it is the only value that is *not equal to itself*. `np.nan == np.nan` returns `False`. That is why you can never test for missingness with `x == np.nan`; you must use a dedicated predicate like `np.isnan(x)` (or Pandas' `isnull`, below). The same quirk is why we need special tools to even *find* the holes.

4 How Pandas unifies `None` and `NaN`

Pandas papers over the two sentinels and lets you use them almost interchangeably. Where it can, it will quietly convert `None` to `NaN` so that data lands in a fast native array:

```
s = pd.Series([1, np.nan, 2, None])
print(s)
```

```
0    1.0
1    NaN
```

```
2    2.0
3    NaN
dtype: float64
```

Notice that the **None** we passed in came back out as NaN, and the whole Series is float64. To make that possible, Pandas sometimes has to change the dtype of your data — a behavior called **upcasting**.

4.1 Upcasting: the price of inserting a NaN

Suppose you have a clean integer Series and you set one element to missing:

```
s = pd.Series([1, 2, 3, 4])
print(s.dtype)         # → int64
s[1] = np.nan
print(s)
print(s.dtype)         # → float64
```

```
int64
0    1.0
1    NaN
2    3.0
3    4.0
dtype: float64
```

Key Idea

Why did an integer column become float? Because NaN is a *floating-point* sentinel — there is no such bit pattern in the integer world. A native int64 array literally has no way to represent "missing." So the moment a NaN must enter an integer column, Pandas *upcasts* the whole column to float64, where NaN is a legal value. Your 1, 2, 3, 4 become 1.0, 2.0, 3.0, 4.0 as a side effect.

Different dtypes upcast differently. The table below summarizes the rules Pandas (in the NumPy-backed model the book describes) applies when a missing value is introduced.

Original dtype	Upcasts to	Sentinel used
float64	float64 (unchanged)	NaN
int64	float64	NaN
bool	**object**	**None** or NaN
object	**object** (unchanged)	**None** or NaN

The pattern: a **float** column already has room for NaN, so it is untouched. An **int** column must climb to **float**. A **bool** column cannot become a float without losing its meaning, so it falls all the way back to slow **object**. An **object** column is already maximally permissive and stays put.

Common Pitfall

Upcasting is silent. If you depend on a column staying `int64` (for an index, a join key, or memory budgeting) and a single missing value sneaks in, the dtype flips to `float64` underneath you and your integers become floats like `3.0`. Always check `df.dtypes` after any operation that could introduce a hole — a merge, a reindex, a `concat`.

5 Operating on null values

Pandas gives a small, consistent vocabulary for detecting, removing, and filling missing data. These four methods cover almost everything you will do day to day.

5.1 Detecting nulls: `isnull` and `notnull`

Both return a boolean mask the same shape as the input — `isnull` flags the holes, `notnull` flags the present values. They work on `Series` and `DataFrame` alike.

```
data = pd.Series([1, np.nan, 'hello', None])
print(data.isnull())
```

```
0    False
1     True
2    False
3     True
dtype: bool
```

Because the result is a boolean mask, you can use it directly to filter — this is the idiom underneath everything else:

```
print(data[data.notnull()])
```

```
0        1
2    hello
dtype: object
```

Python Refresher

`isna` is an exact alias for `isnull`, and `notna` for `notnull` — they do the same thing, and which you see is mostly a matter of an author's taste. Both detect `None` *and* `NaN` (and, in modern Pandas, `NaT` for missing datetimes and `pd.NA`).

5.2 Removing nulls: `dropna`

On a `Series`, `dropna()` simply discards the missing entries. On a `DataFrame` the choice is richer, because you cannot drop a single cell — you must drop a whole *row* or a whole *column*. Consider:

```
df = pd.DataFrame([[1,      np.nan, 2],
```

```
                    [2,      3,      5],
                    [np.nan, 4,      6]])
print(df)
```

```
     0    1  2
0  1.0  NaN  2
1  2.0  3.0  5
2  NaN  4.0  6
```

The `axis` argument decides the unit of removal, and `how` decides the threshold:

```
df.dropna()                  # axis=0 default: drop any ROW containing a NaN
df.dropna(axis=1)            # drop any COLUMN containing a NaN
df.dropna(axis=1, how='all')  # drop a column only if ALL its values are
    NaN
df.dropna(thresh=3)          # keep rows with ≥ 3 non-null values
```

Intuition

Read `axis` as "the axis along which I am moving the blade." `axis=0` (the default) slices off *rows*; `axis=1` slices off *columns*. Then `how='any'` (the default) is aggressive — one hole and the whole row/column goes — while `how='all'` is lenient, removing only the fully-empty ones. When neither is precise enough, `thresh=n` keeps anything with at least `n` real values, giving you a tunable middle ground.

For the frame above, `df.dropna()` leaves only row 1 (the one row with no holes):

```
     0    1  2
1  2.0  3.0  5
```

Common Pitfall

`dropna` returns a *new* object by default — it does not modify your frame in place. If you write `df.dropna()` on a line by itself and then keep using `df`, nothing changed. Capture the result (`df = df.dropna()`) or, with care, pass `inplace=True`. The same caveat applies to `fillna`.

5.3 Filling nulls: `fillna`

Often you would rather *repair* the holes than delete the rows that contain them. `fillna` replaces each missing value, either with a constant or by copying a neighbor.

```
data = pd.Series([1, np.nan, 2, None, 3], index=list('abcde'))
print(data.fillna(0))        # constant fill
```

```
a    1.0
b    0.0
c    2.0
```

```
d    0.0
e    3.0
dtype: float64
```

Two directional fills copy an adjacent value into the gap — forward-fill carries the *previous* value down, back-fill pulls the *next* value up:

```
data.ffill()   # forward-fill: propagate last valid value forward
data.bfill()   # back-fill: propagate next valid value backward
```

```
# ffill:                  # bfill:
a    1.0                  a    1.0
b    1.0                  b    2.0
c    2.0                  c    2.0
d    2.0                  d    3.0
e    3.0                  e    3.0
```

On a `DataFrame` the same fills take an `axis` (fill down columns vs. across rows), and a `limit` caps how many consecutive holes a single fill will bridge:

```
df.fillna(method='ffill', axis=1)   # carry values rightward across each
    row
df.ffill(limit=1)                   # fill at most 1 consecutive NaN
```

Python Refresher

The book uses the keyword form `fillna(method='ffill')` and `fillna(method='bfill')`. Modern Pandas exposes the dedicated methods `ffill()` and `bfill()` shown above and has deprecated the `method=` keyword; the behavior is identical. If a forward-fill meets a leading `NaN` with no prior value to copy, that entry simply stays `NaN`.

6 A note on modern Pandas: `pd.NA` and nullable dtypes

Background & Context

VanderPlas's text predates a significant addition to the library, and it is worth knowing because it removes the most annoying limitation above — the forced `int`-to-`float` upcast.
Modern Pandas introduces a single, type-agnostic missing-value indicator, `pd.NA`, together with a family of **nullable dtypes**: `Int64`, `boolean`, and `string` (note the capital `I` in `Int64` — that capitalization is how you tell the nullable extension type apart from the old `int64`). Internally these use something closer to the **mask** approach from the very first box: the values and a separate validity mask, so a column can be a genuine integer column *and* hold missing values at the same time.

- `pd.Series([1, 2, None], dtype="Int64")` stays an integer column; the hole shows as `<NA>`, and there is *no* upcast to float.
- `df.convert_dtypes()` inspects a frame and migrates its columns to the best available

nullable types automatically.
- `isna`/`notna` recognize `pd.NA` just as they do `NaN` and `None`.

We keep the rest of these notes grounded in the `None`/`NaN` model, because that is what you will see in the book, in older code, and in the default behavior of most NumPy-backed columns. But when you find yourself fighting an unwanted upcast in real work, nullable dtypes are the modern escape hatch.

Section recap

- Two strategies for missing data: a separate **mask** (extra memory) vs. an in-band **sentinel** (costs a representable value, type-specific). NumPy-backed Pandas historically used sentinels.
- `None` is a Python object sentinel: it forces `dtype=object`, runs slowly, and makes `sum()` raise `TypeError` because `int + None` is undefined.
- `NaN` is a native floating-point sentinel: fast, but *contagious* — any arithmetic touching it yields `NaN`. Use `np.nansum`, `np.nanmean`, etc. to skip holes. Also, `NaN != NaN`.
- Pandas interchanges `None` and `NaN`, and **upcasts** dtypes to fit a `NaN`: `int`→`float`, `bool`→`object`, `float` and `object` unchanged.
- Core toolkit: `isnull`/`notnull` (masks), `dropna` (`axis`, `how`, `thresh`), `fillna` (scalar, `ffill`, `bfill`, `axis`, `limit`). Most return a new object unless `inplace=True`.
- Modern Pandas adds `pd.NA` and nullable dtypes (`Int64`, `boolean`, `string`) so integers can hold missing values without upcasting — helpful context the book predates.